CEN 571/CSE 598/CSE 494 Lab 4

Adaptive Dataflow Design using AMD's AI Engines

1. Objective

This lab is intended to help you:

1 | Learn to program a Coarse Grained Reconfigurable Array (CGRA) type of accelerator

- Learn how to use AMD Vitis Unified IDE for AI Engine graph and kernel development

2 | Vectorize kernels to reduce the execution latency and improve performance

- Transform the unoptimized scalar implementations of the kernel to an efficient vector implementation to improve the performance

3 | Explore performance and scaling trade-offs

- Distribute your workload across multiple processing elements
- Experiment with scalar vs. vector implementations, single vs. double buffering, and multiple processing element mapping across AMD AI Engine cores

2. Background

Steganography is the practice of concealing a message or file within another message or file. In this lab, you will take on the role of a digital forensics engineer tasked with uncovering hidden information. You are provided with an encrypted image and a corresponding key image as inputs. The objective is to generate an output image that reveals the concealed message. Figure *1* shows a high level overview of the task. Figure *2* shows the steps involved in the decoding process. First, decrypting the image using a specific security key through XOR-based decryption, and second, applying image processing filters such as thresholding to enhance readability and clearly display the hidden content.

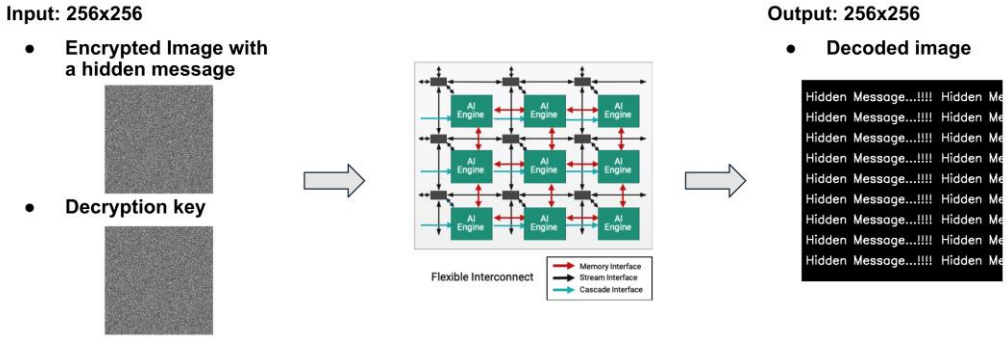


Figure 1: High Level Overview

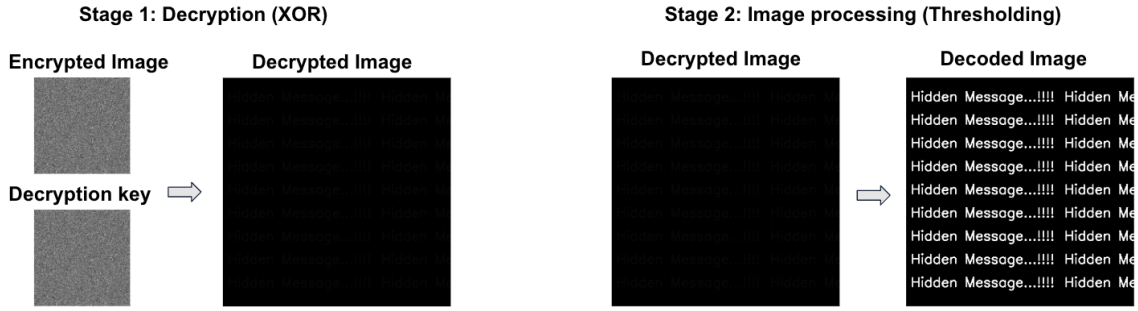


Figure 2: Steps involved in decoding

The challenge lies in the fact that standard processors are often too slow to perform real-time decryption of high-resolution data streams. To address this, you will leverage AMD's AI Engines (AIEs) to optimize the processing pipeline for maximum throughput. The AIE programming model consists of two primary components: the Kernel, which executes code within each individual AIE, and the Graph, which defines the interconnections and data flow between the AIEs.

3. Handout Overview

This handout provides a step-by-step tutorial on setting up a workspace in the Vitis Unified IDE using the provided base code files. It guides you through the compilation and simulation of the AIE design. To help you get started, we provide a resource package (lab4.zip) that includes the following assets:

Input images: An encrypted input image (stego_image.png) and the decryption key (key_image.png)

Baseline source code files located in aie_source folder:

- **Kernels located in aie_source/kernels folder:** Baseline (Scalar/Non-vectorized) for the decryption and image processing kernels
    - "decrypt.cc" :- Contains a baseline scalar implementation for decryption
    - "thresholding.cc" :- Contains a scalar implementation for thresholding
- **Baseline Graph:** A simple connectivity graph ("aie_top.h") file defining the data flow between AI Es and IO ports.
- **Host**: The AIE simulator host code ("aie_top_all.cpp"). This is used to initialize and run the AIE design. It also initiates the data transfer from host to AIEs and back.
- **"png_utils":** Helper functions to read/write a png file

4. Using Vitis Unified IDE

AMD Vitis Unified IDE is the software development design flow used to compile, map and debug a user design on AMD's Versal devices containing AI Engine arrays. While traditional FPGA tools synthesize hardware description languages, the Vitis AI Engine flow utilizes a software centric approach. The process begins with compiling user designs written in C/C++, which define compute kernels and Adaptive Data Flow (ADF) graphs that describe the data movement and execution flow within the AIE architecture.

The kernel handles the core compute on an AIE core while the ADF graph handles the connections between specific AIE cores within the AI Engine array and the Input/Outputs. For simulation, Vitis Unified IDE offers fast x86 functional simulator (X86SIM), a cycle-approximate AI Engine simulator (AIESIM) with system-level event tracing and performance profiling. You can find the documentation for AI Engine development, such as the AIE hardware architecture manual (AM020), AIE Kernel and Graph programming guide (UG1079), and AI Engine Tools and Flows user guide (UG1076), provided by AMD. We will use AI Engine API's for writing our kernels. These API's make the processes of designing the kernel simple. Although we will give a short tutorial on how to use the Vitis Unified IDE in the context of this assignment, you might want to invest some time skimming through the contents of these documents for more details. You can also find several AI Engine programming examples from Vitis-Tutorials GitHub repository.

Follow these steps to set up your project, compile, and simulate your design:

4.1 Setting up Vitis Unified IDE workspace and project

Launch a Sol desktop session with at least **16 Cores and 32 GB of RAM**. Export the **XILINXD_LICENSE_FILE** environment variable to set the license server, then source Vitis. Please use the following commands on a SOL cluster node:

```
export XILINXD_LICENSE_FILE=2100@en4228283l.scai.dhcp.asu.edu
source /data/courses/class_cse494598cen571spring2026_aaror112/Vitis_2024_1/Vitis/2024.1/settings64.sh
```

Launch Vitis using the "**vitis**" command. This should open a GUI as shown in Figure *3*.

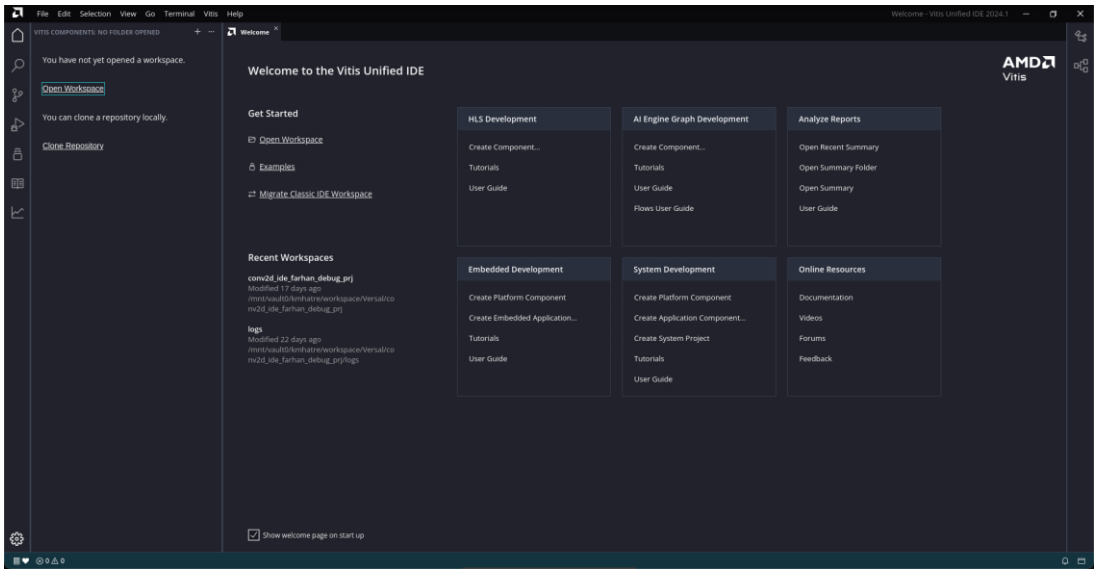


Figure 3: Vitis Unified IDE GUI

Create a new folder (directory) on your SOL cluster node; this will be your **workspace** for Lab 4. Download the lab4.zip archive from Canvas, and transfer the archive to your home directory on the RC Sol Cluster and unzip the lab4.zip archive file in the Sol Cluster. Then, in Vitis, go to **File → Open Workspace** (see Figure 4), choose that folder, and click **Open**.

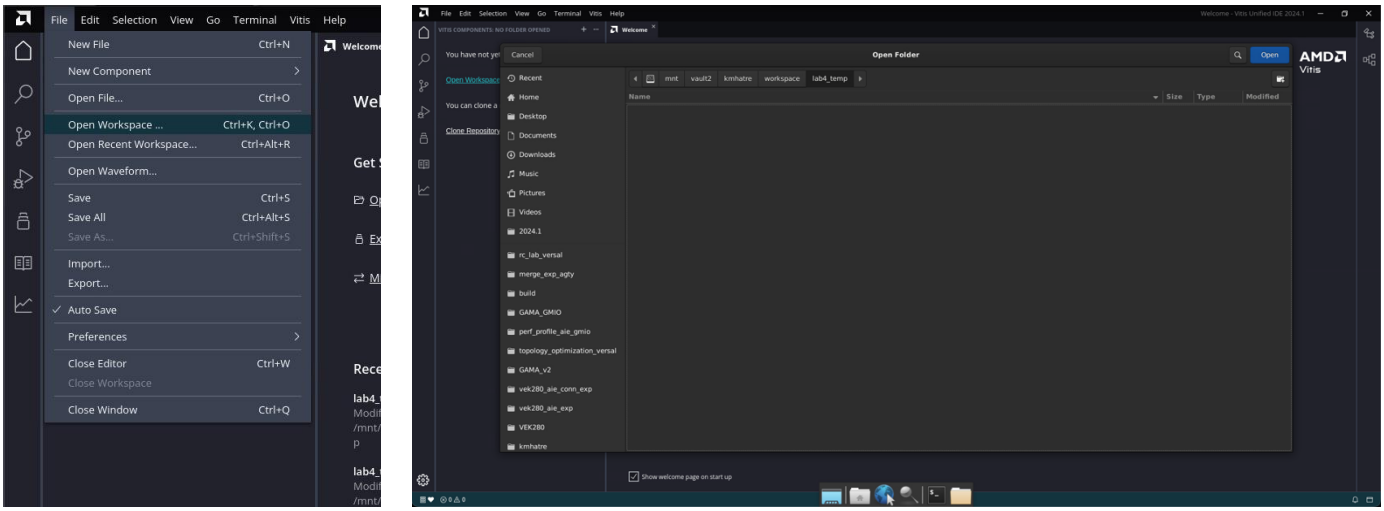


Figure 4: Workspace creation

The IDE window will look as shown in Figure *5*.

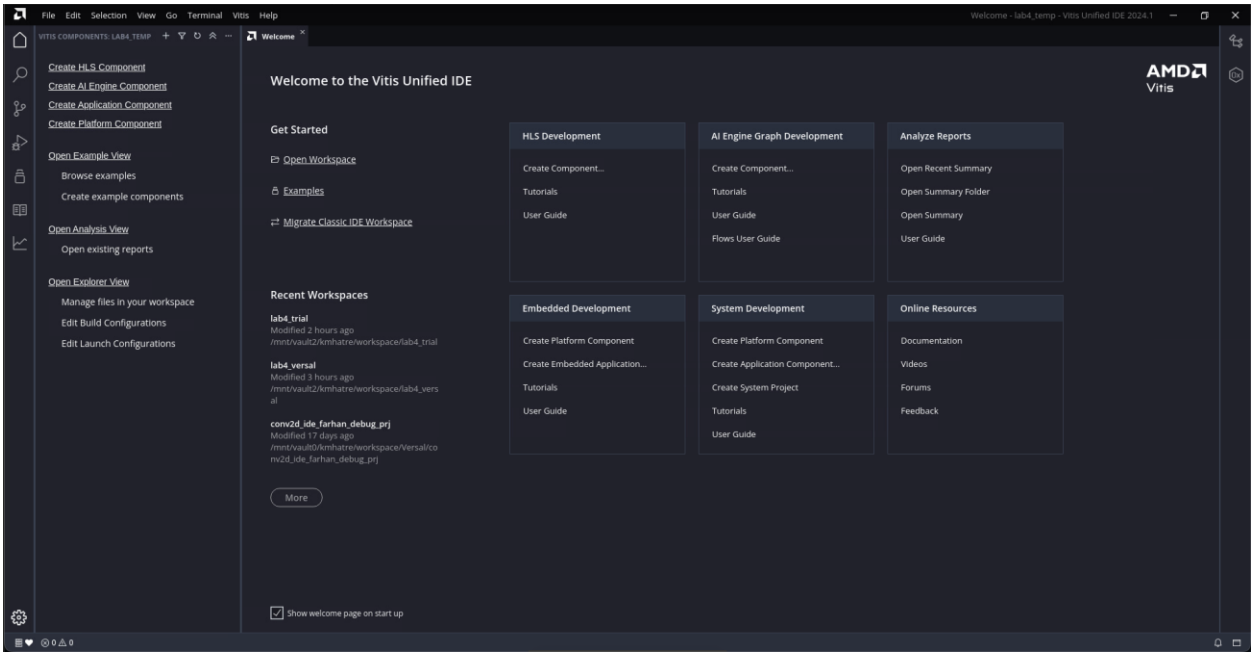


Figure 5: Vitis IDE with a workspace open

In the left-side menu, click **Create AI Engine Component**. A setup window will open (shown in *Figure 6*). Since this lab focuses only on AI Engine (AIE), make sure you are creating an AIE component. Type a name for your component, then click **Next**.

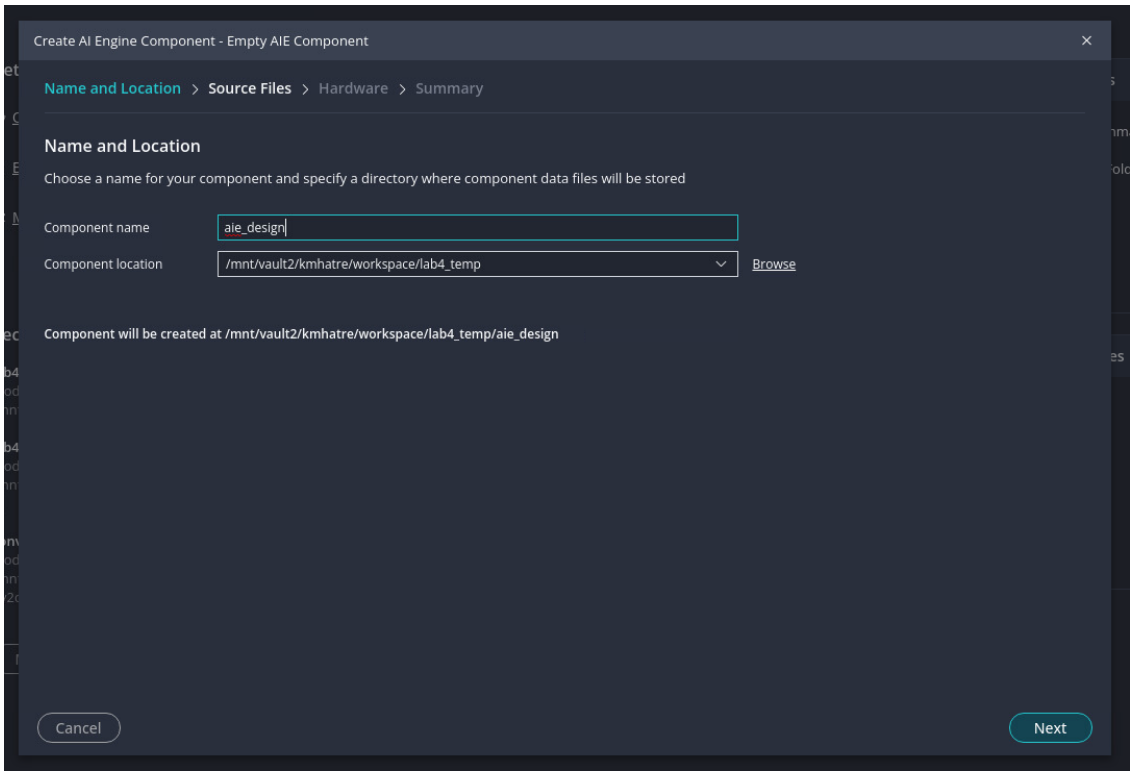


Figure 6: Provide a name to the design

In the next screen (see *Figure 7*), add the **source directory**. Select the folder that contains the lab files you were given. After you add it, Vitis will copy those files into your workspace under the **aie_design** component.

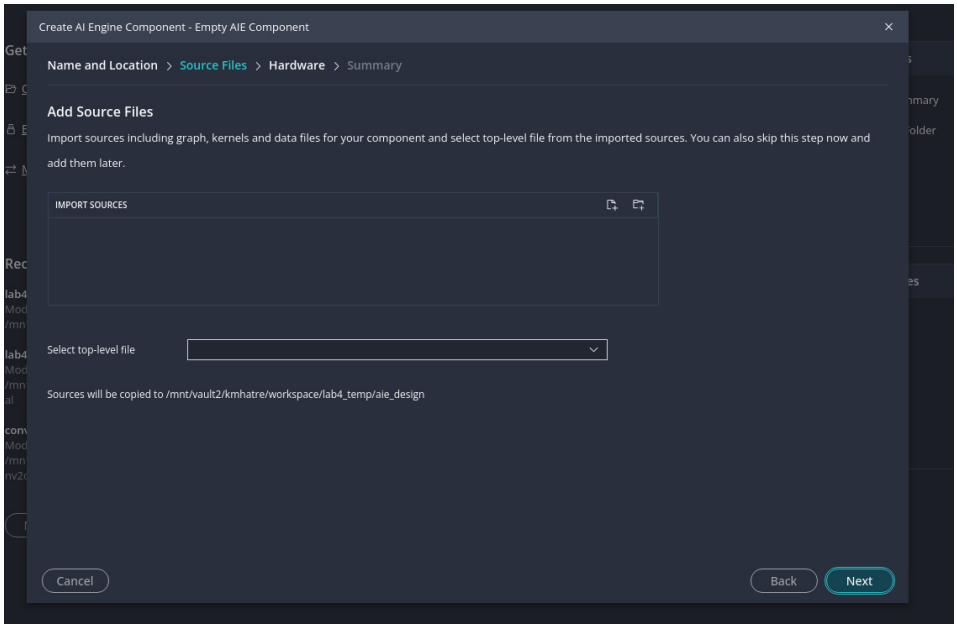


Figure 7: Add source files

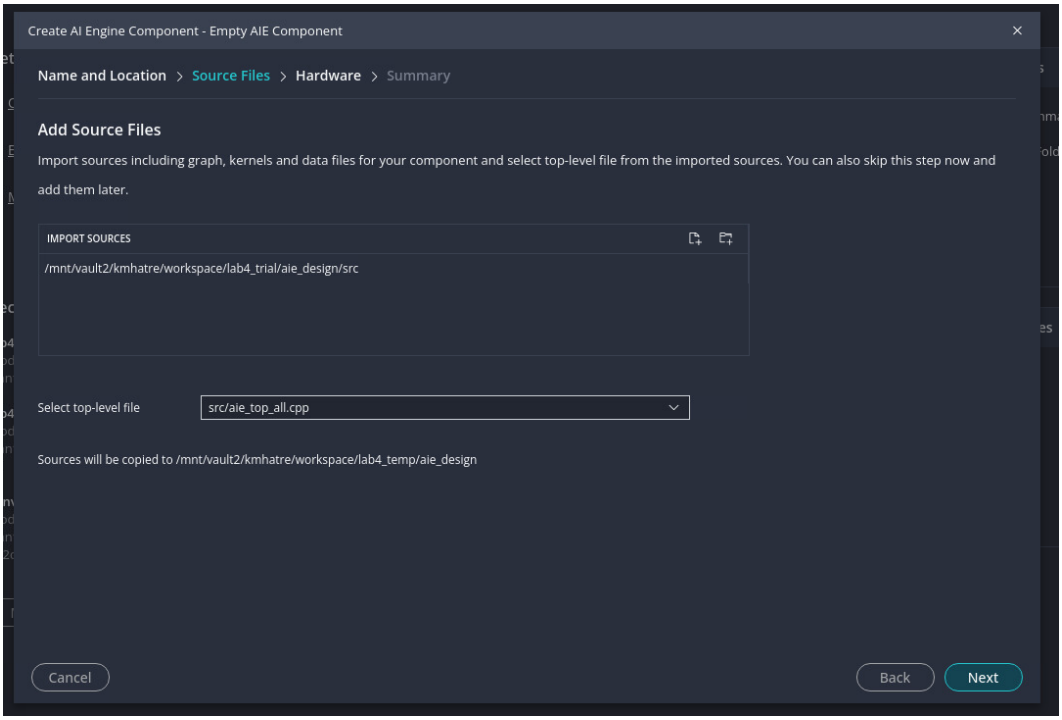


Figure 8: Select the right top file

Next, select the top-level file **aie_top_all.cpp** (shown in Figure 8). This step is important, if you choose the wrong top file, compilation may fail.

Click **Next** to choose the base platform (shown in Figure 9). This selection is like choosing an FPGA device/board in Vivado, it determines the target device, AIE version, and how many AIE tiles are available. Select **Xilinx_vek280_base_202410_1**.

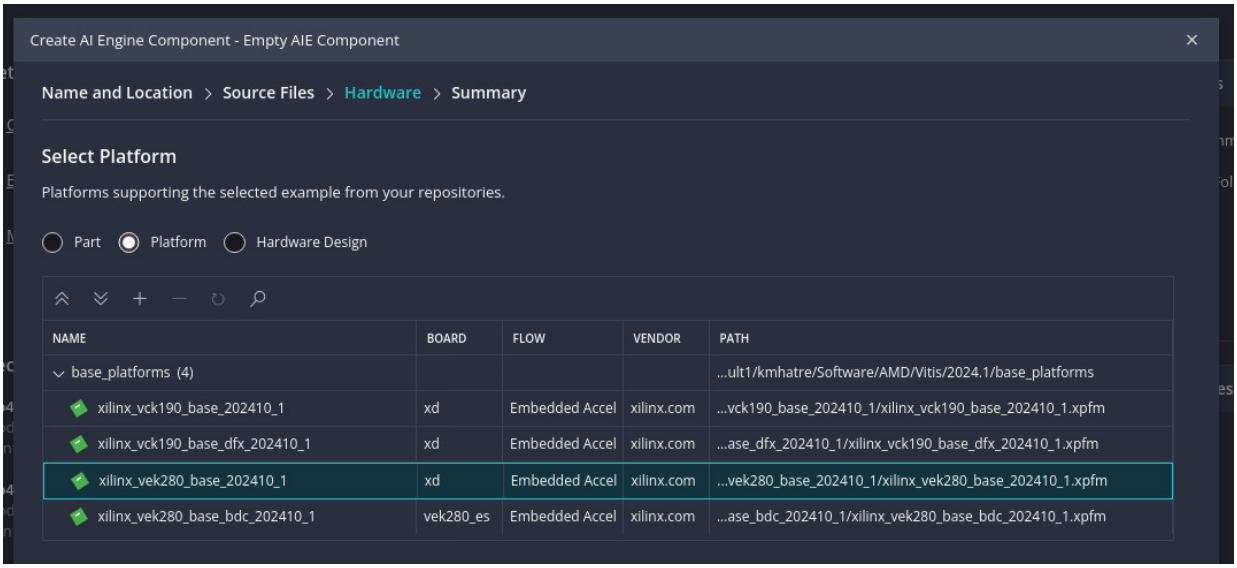


Figure 9: Select the right Platform file

Click **Finish**. Vitis will create the project using the source files and the platform you selected. You should now see the component opened in the IDE, similar to Figure 10.

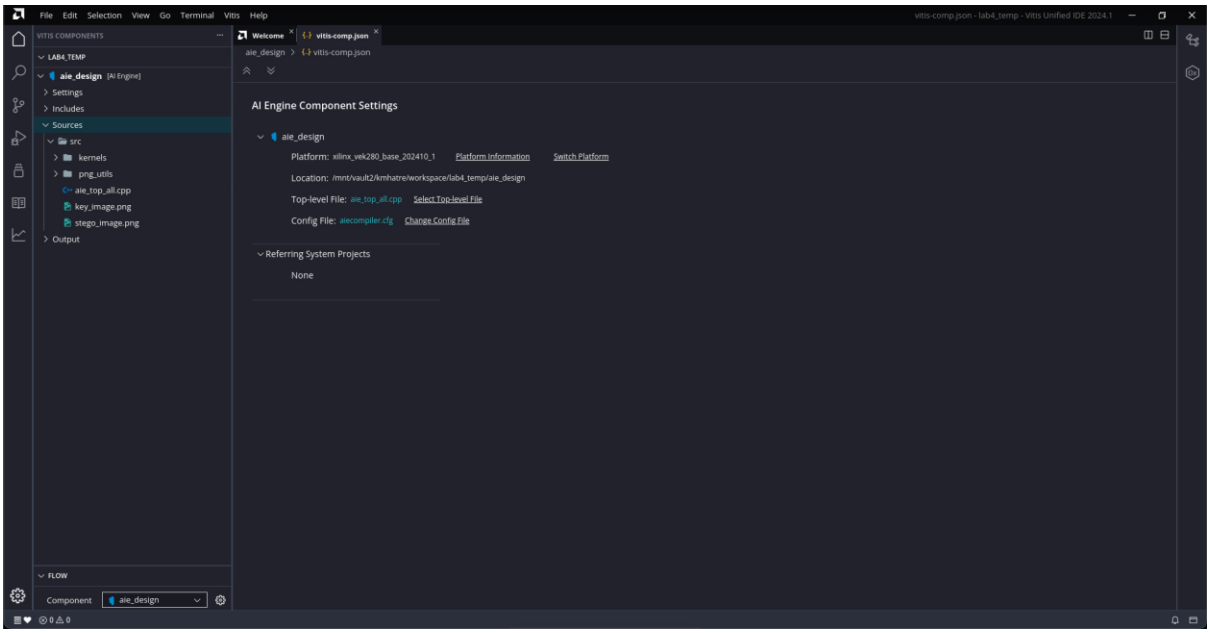


Figure 10: IDE with a component open

4.2 Configuring compilation and simulation settings

On the left side, you should see the source files that were added when you created the project.

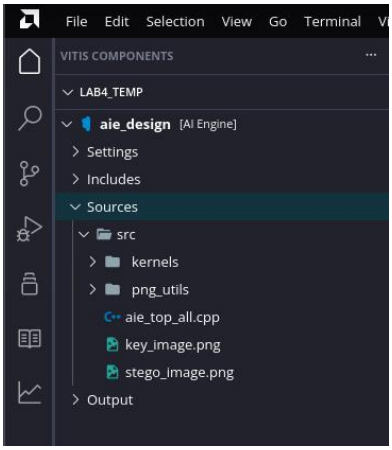


Figure 11: Source files

Expand **Settings** as shown in Figure 12. You should see two important files related to the AIE compiler and simulator:

- “aiecompiler.cfg”:- Has information/settings regarding the AIE compiler
- “launch.json”:- Has simulation related settings

Open **aiecompiler.cfg**. Click **Add item**, then add these include directories (one at a time):

- "src/kernels"
- "src/png_utils"

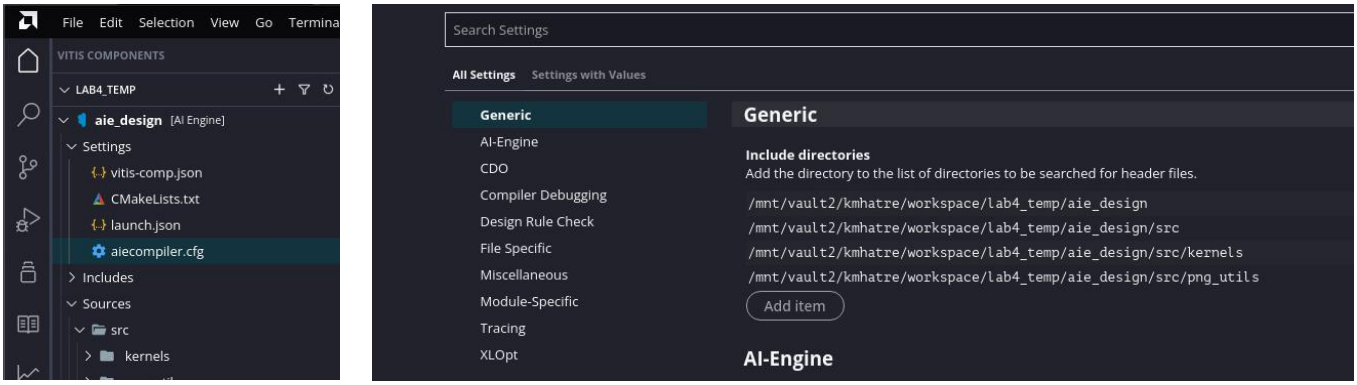


Figure 12: Adding includes

Next, open **launch.json** to configure simulation settings. Turn on the **Trace** and **Profile** options, as shown in Figure 13.

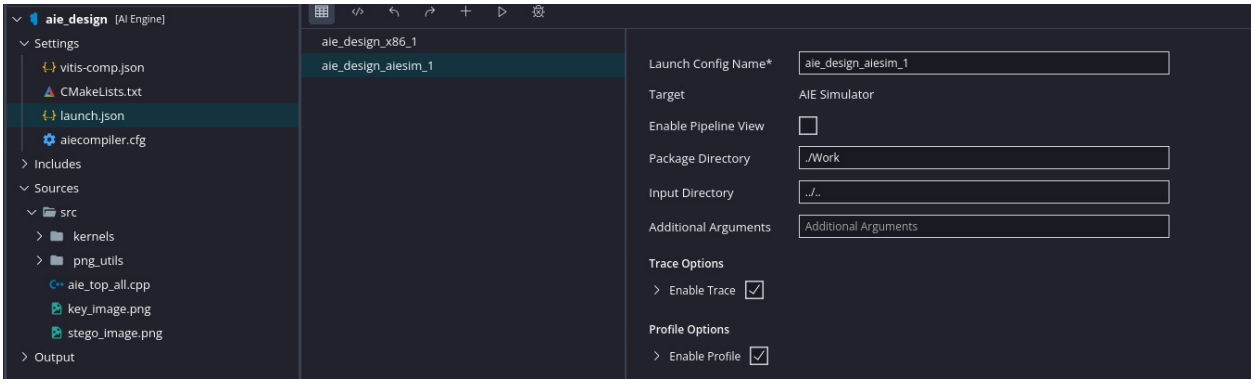


Figure 13: Enable tracing and profile in settings

That's it for setup, you can now compile and simulate the design.

4.3 Compilation and Simulation

In this lab, you will use two types of compilation/simulation: **x86 simulation** and **AI Engine simulation (aiesim)**. We use simulation because we only have one physical Versal board in the lab, so it cannot be shared by a large group at the same time.

x86 simulation: This is the fastest option. Use it first to test, debug, and verify that your AIE kernels and graph are functionally correct. Because it uses a simplified model, it does **not** provide accurate timing, resource estimates, or performance results. The simulation runs as x86 threads on your machine, so it also supports lots of **printf()** debugging and step-by-step debugging with **GDB**.

AI Engine Simulator (aiesim): This simulator models the timing, resources, and memory behavior of the AIE array. Run **aiesim** when you need cycle counts and a more realistic performance estimate. It can generate profiling data and VCD trace files that help you measure throughput, find stalls (for example, stream stalls), and identify bottlenecks. Note: aiesim does not include the latency from DRAM/NoC to the interface tile, so it does not capture DRAM access overhead.

In Vitis, look at the bottom-left pane for the buttons to **Build** and **Run** both x86 simulation and aiesim. Figure 14 highlights these options.

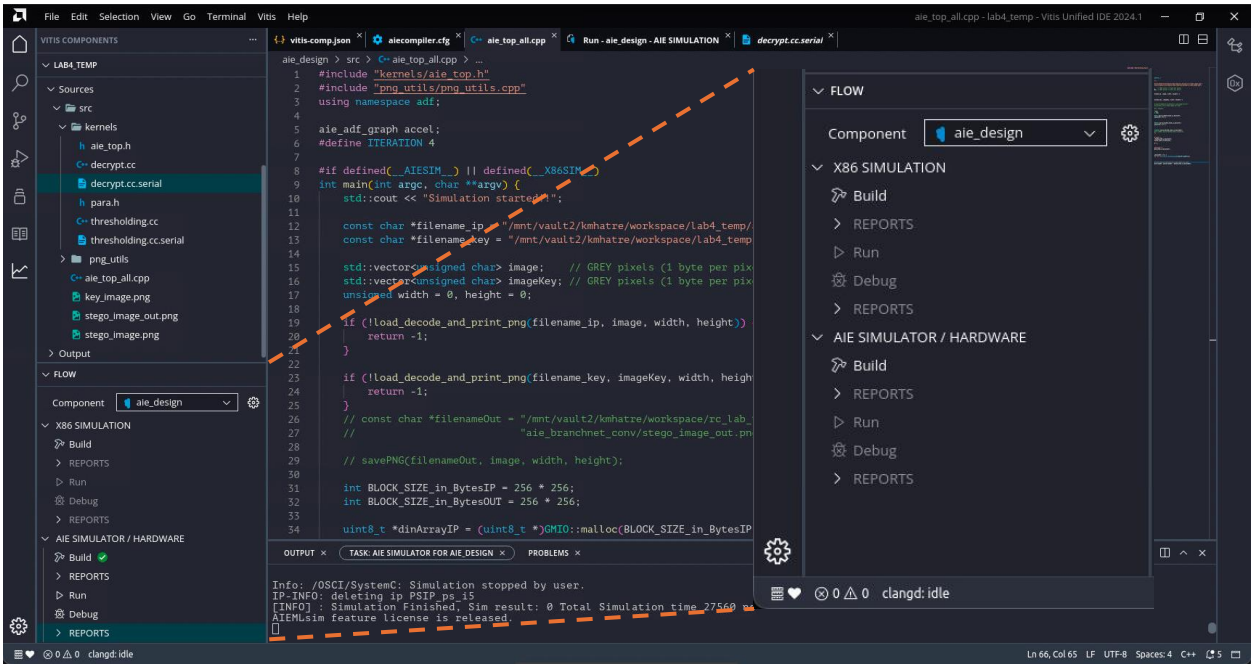


Figure 14: Compilation and Simulation

In the file explorer on the left, you should see two input images: **stego_image.png** and **key_image.png**. These files are used as inputs by **aie_top_all.cpp**.

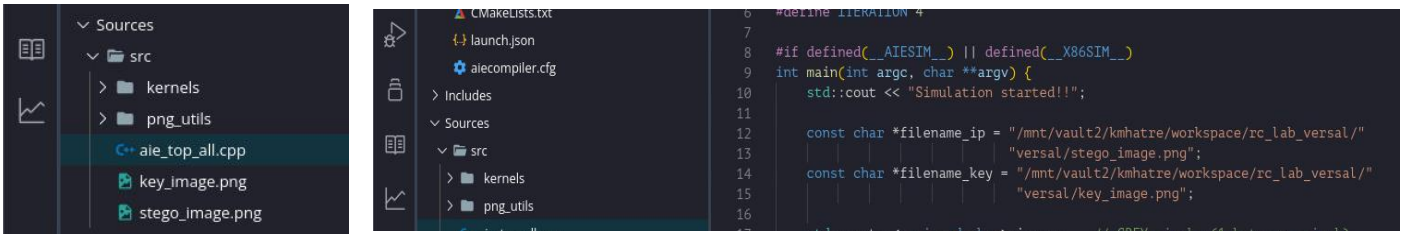


Figure 15: Input images and the path setting

Open **aie_top_all.cpp** and update the code to use the **absolute paths** to **stego_image.png** and **key_image.png** (see Figure 15). Moreover, update the output path (*filename_out) to write the output image in the specific directory. Then, under **x86simulation**, click **Build**. Wait for the build to finish. When it completes successfully, you should see “Compilation complete” and “ERRORS:0”. You should also see a green check mark next to **Build** (Figure 16).

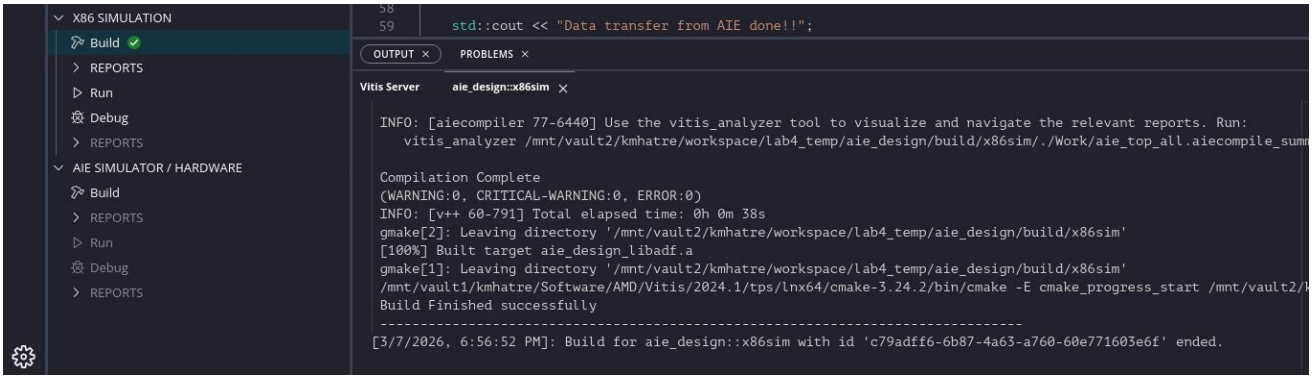


Figure 16: x86 compilation completed

After the build succeeds, run the simulation. Click **Run** as shown in Figure 17.

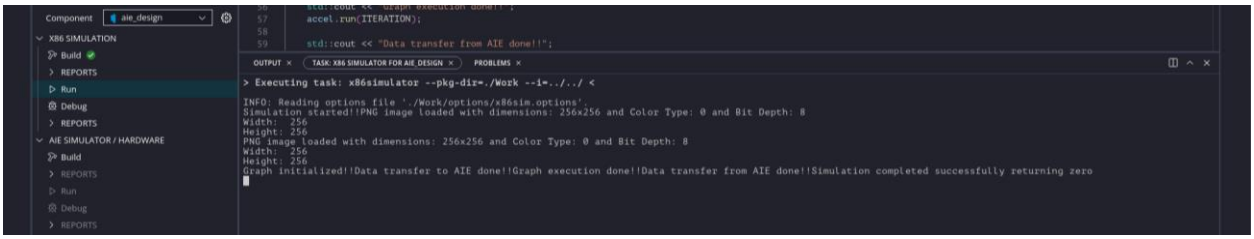


Figure 17: x86 simulation

Wait for the run to finish. The simulator will write the output image to the specified directory as **decrypted_image_out.png**. Open this file to view the decrypted message (in our example, the message is “Reconfig lab hi”).

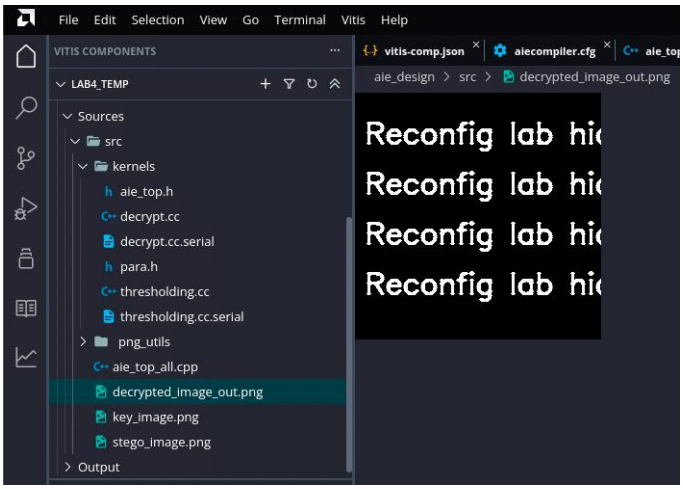


Figure 18: The decrypted output image

At this point, you have successfully run **x86 simulation**. Remember: x86 simulation checks functional correctness, but it is not meant for accurate performance. Next, you will run **aiesim** to measure execution cycles on the AIE model.

Under the **AIE SIMULATOR/HARDWARE** section, click **Build**. This build usually takes longer than the x86 build. When it finishes, you should see a summary like "(WARNING:0, CRITICAL-WARNING:0, ERROR:0)". Then click **Run** to start aiesim. aiesim can take significantly longer than x86 simulation. After it completes, you will see more entries appear under the **REPORTS** section.

4.4 Analyzing the Reports

The **REPORTS** section contains useful views for understanding your design, including the graph connectivity, kernel placement, trace, and profiling/performance results.

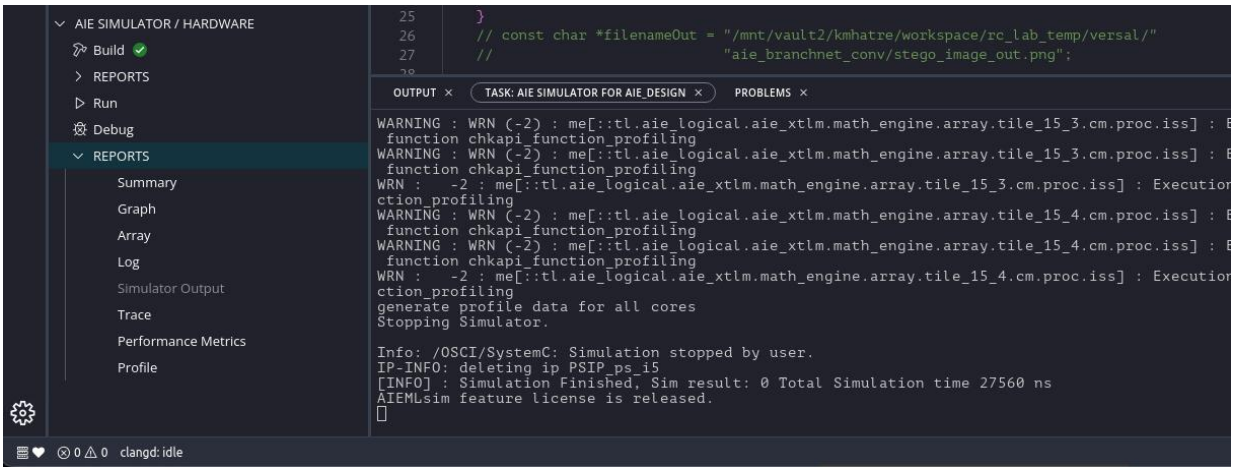


Figure 19: Reports section after AIE simulation

Figure 19 shows the available report views:

- **Summary**: Overall status plus resource utilization.
- **Graph**: Logical connectivity between kernels (how data flows).
- **Array View**: Physical placement of kernels and buffers on the AIE array.
- **Trace**, **Performance Metrics**, and **Profile**: Performance analysis tools to help you evaluate and debug bottlenecks.

Graph view

Click **Graph** to view a diagram of how your AIE kernels connect. Figure 20 shows an example connectivity graph. The square in the center (labeled (**1**)) is an AIE kernel. Because this is a **logical** graph, it does not tell you the physical tile location. You should see two inputs entering the graph and one output leaving it. The **stream memory** block (labeled (**2**)) represents buffers allocated in AI Engine local memory. For AIE-ML, each tile has 64 KB of local memory. The endpoints labeled (**3**) are **GMIO** connections, which move data between global DDR memory and the AIE array through the interface tile.

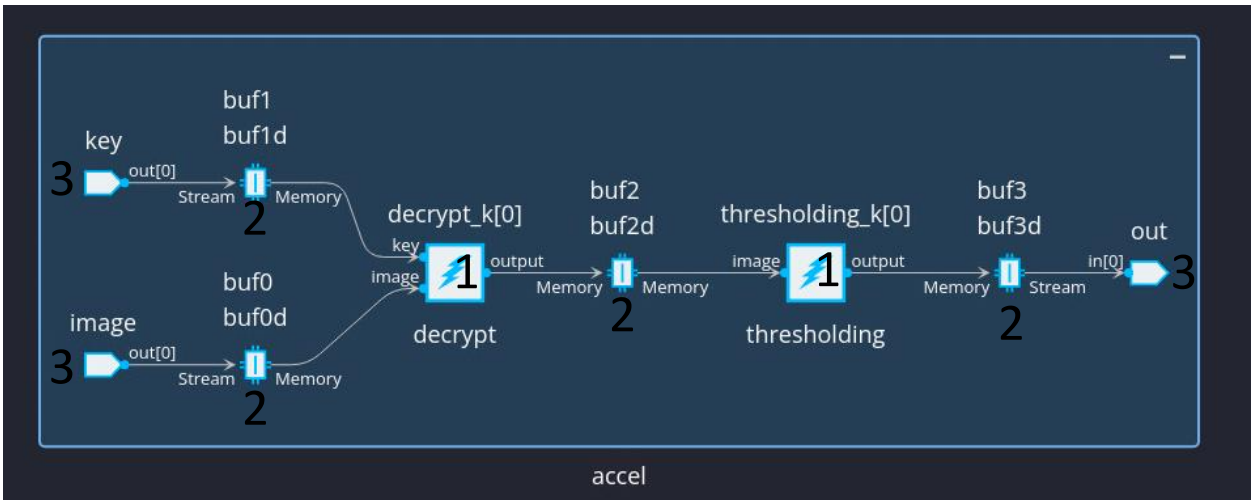


Figure 20: Graph view showing AIE kernel connectivity

Array view

Next, click **Array** to see where the design is placed on the physical AIE array. This view shows:

- The physical tile location of each kernel.
- The placement of local buffers used by each kernel.
- Any buffers placed in neighboring tiles (neighboring AIE tiles can share memory).
- GMIO ports mapped near the interface tile row.

Figure 21 shows the physical mapping for the baseline design.

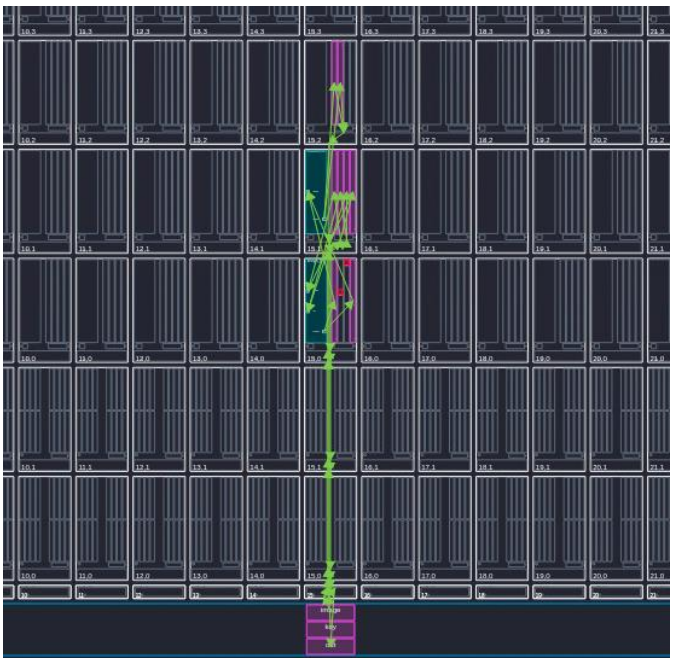


Figure 21: Array view

Now that you have seen the design graph and kernel placement, it's time to look at performance.

Performance profiling

This section is critical for understanding and evaluating design performance. Figure 22 highlights the primary analysis of views - **Trace**, **Performance Metrics**, and **Profile**, which together provide insight into execution behavior and efficiency.

- **Trace** view presents a waveform of kernel execution
- **Performance Metrics** reports stall and utilization statistics
- **Profile** summarizes execution cycles for individual kernels and the overall graph across AIE tiles.

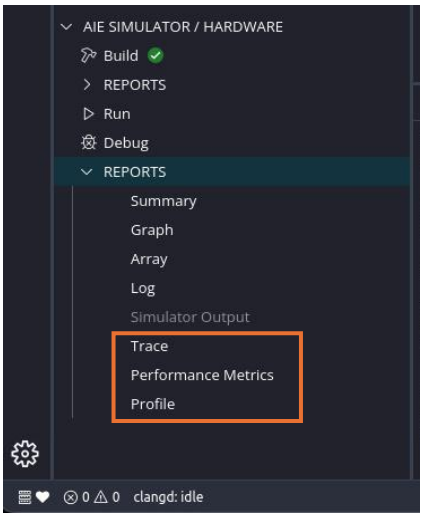


Figure 22: Performance profile

Click **Profile** to open profiling results. In Figure 23, on the left you can see results for each AIE Tile.

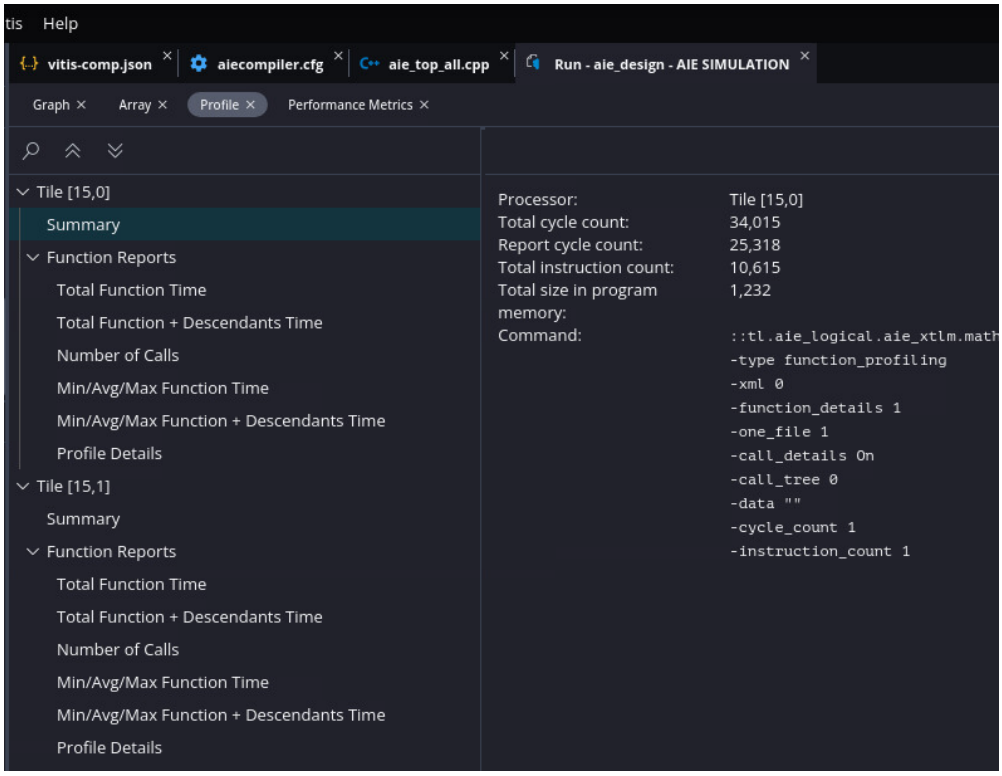


Figure 23: Summary section of the profile

Click **Total Function Time** (Figure 24) to see the execution time for each kernel. This is the main view you will use to compare performance as you optimize your code. The highlighted part shows the execution

cycles and number of calls for the decrypt kernel, it took **10304** cycles for all **4** calls, thus single call is 2576 cycles. Note that these values are examples and they are not the real values you should report.

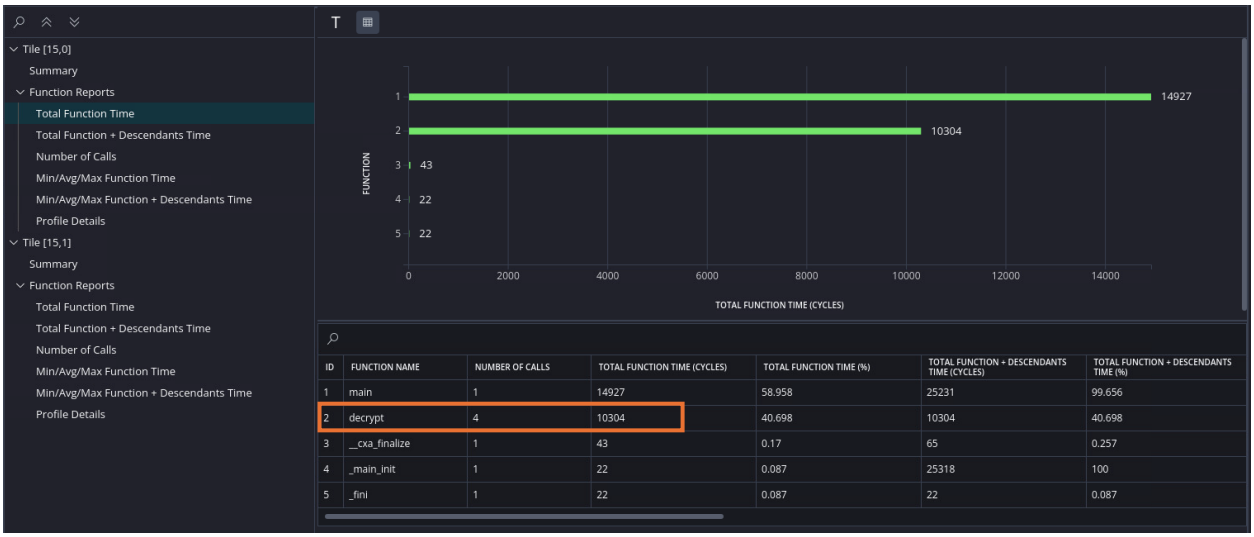


Figure 24: Total Function Time section

To view the execution timeline, click **Trace** in the left panel (Figure 25).

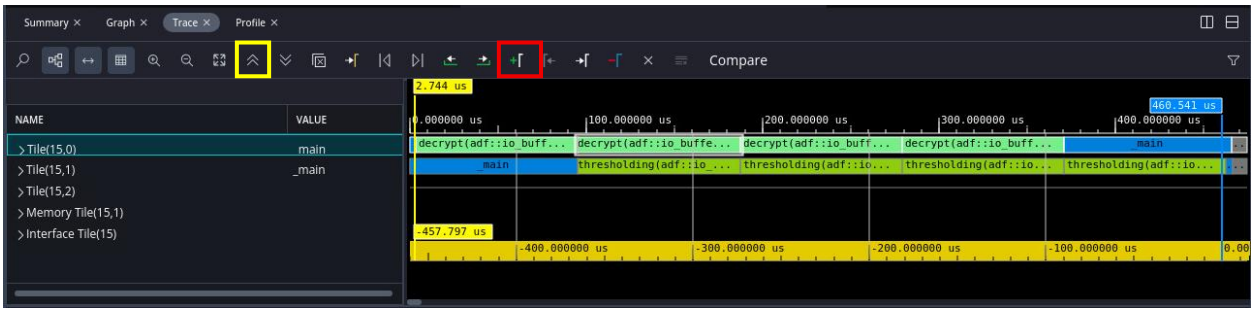


Figure 25: Execution VCD Trace

The trace shows a timeline of the full application run. Use it to separate **data transfer time** from **compute time** for each kernel. You can also place markers to measure the time between specific events.

Note: *The trace view can contain many signals, which may make it difficult to interpret. First, use the filter in the top-right corner of the trace window to display only the signals relevant to your analysis. Alternatively, you can collapse all signals as shown in Figure 25 (using the collapse button highlighted in yellow). This view shows only kernel execution activity and is recommended when measuring execution time. To measure latency, use markers. A marker appears as a vertical line on the trace and indicates a specific timestamp. The default yellow marker marks the start of the trace, and the timestamp at the top of the marker shows its exact time (for example, 2.744 µs). You can add additional markers by clicking the Add Marker button (highlighted by a red box). In Figure 25, a second marker (blue) has been added at the end of execution, marking a time of 469 µs. When two markers are present, the time difference between them is displayed at the bottom of the 1st marker (for example, 457.797 µs). Use this difference to measure*

execution latency. For all experiments, measure time from the start of the first kernel to the end of the last kernel, and apply this measurement method consistently.

5. AIE Kernel and Graph design

The programing model of the AIE compiler has 2 different components.

- Kernel: The kernel is where we write the code that is executed inside an AIE
- Graph: The graph defines the connectivity of the kernels

We provide a sample scalar kernel ( ) as below.

```
1  void decrypt(adf::input_buffer<uint8_t> &__restrict image,
2               adf::input_buffer<uint8_t> &__restrict key,
3               adf::output_buffer<uint8_t> &__restrict output) {
4
5      // Size of image
6      constexpr int num_elems = IP_IMG_H * IP_IMG_W;
7
8      // Set the input pointer to read from the memory buffer
9      const uint8_t *__restrict pA = image.data();
10     const uint8_t *__restrict pB = key.data();
11     uint8_t *__restrict pC = output.data();
12
13     // Loop over each element
14     for (int i = 0; i < num_elems; ++i)
15     chess_prepare_for_pipelining
16     {
17         pC[i] = pA[i] ^ pB[i];
18     }
19 }
```

The code above shows a kernel that **decrypts** the image by performing a bitwise **XOR** between the **image** buffer and the **key** buffer. These buffers live in AI Engine memory shown as label 2 in Figure 20: the kernel reads from the two input buffers and writes the decrypted result to the **output** buffer. The directive **chess_prepare_for_pipelining** tells the compiler to pipeline the loop so a new iteration can start every cycle (when possible), which improves throughput. This code snippet shows only the compute kernel. Next, you will look at the **connectivity graph**, which describes how data moves between memory, kernels, and output.

The connectivity of the kernels is defined in a file called **aie_graph.h**

```
1.  class aie_adf_graph : public adf::graph {
2.  public:
3.      input_gmio image;
4.      input_gmio key;
5.      output_gmio out;

6.      kernel decrypt_k[1];
7.      kernel thresholding_k[1];

8.      aie_adf_graph() {
```

```
9.          image = input_gmio::create("image", 64, 10000);
10.         key = input_gmio::create("key", 64, 10000);
11.         out = output_gmio::create("out", 64, 10000);

12.         decrypt_k[0] = kernel::create(decrypt);
13.         source(decrypt_k[0]) = "decrypt.cc";
14.         runtime<ratio>(decrypt_k[0]) = 1;

15.         thresholding_k[0] = kernel::create(thresholding);
16.         source(thresholding_k[0]) = "thresholding.cc";
17.         runtime<ratio>(thresholding_k[0]) = 1;

18.         connect<>(image.out[0], decrypt_k[0].in[0]);
19.         dimensions(decrypt_k[0].in[0]) = {IP_IMG_H * IP_IMG_W};

20.         connect<>(key.out[0], decrypt_k[0].in[1]);
21.         dimensions(decrypt_k[0].in[1]) = {IP_IMG_H * IP_IMG_W};

22.         connect<>(decrypt_k[0].out[0], thresholding_k[0].in[0]);
23.         dimensions(decrypt_k[0].out[0]) = {IP_IMG_H * IP_IMG_W};
24.         dimensions(thresholding_k[0].in[0]) = {IP_IMG_H * IP_IMG_W};

25.         connect<>(thresholding_k[0].out[0], out.in[0]);
26.         dimensions(thresholding_k[0].out[0]) = {IP_IMG_H * IP_IMG_W};
27.     }
28. };
```

To interface with the DRAM or Programmable logic, AMD provides us with 2 types of interfaces.

- PLIO:- This interface connects the AIE to PL
- GMIO:- This interface connects to the DRAM global memory.

In our lab we are going to use GMIO to connect the external memory directly.

Lines 9-11 initialize input_gmio and output_gmio objects to establish external memory-mapped connections between the AI Engine graph and global DDR memory. These create functions define the logical port name, the memory transaction burst length (64 bytes), and the expected bandwidth (10000 MB/s) for the image and key data transfers. **Lines 12 and 15** use kernel::create() to instantiate the decrypt and thresholding compute nodes, which act as the fundamental building blocks of the data flow graph. **Lines 13 and 16** use the source() function to specify the external C++ source files (decrypt.cc and thresholding.cc) that contain the actual computational logic for these kernels. The connect<>() statements (e.g., **Lines 18, 20, 22, 25**) define the topology and connectivity of the design, equivalent to the nets in a hardware data flow graph. **Lines 18 and 20** specifically route the image and key GMIO memory inputs directly into the first and second input ports (in and in) of the decrypt_k kernel. **Line 22** wires the output port of the decrypt kernel directly to the input port of the thresholding kernel, defining the sequential execution pipeline. **Line 25** links the final processed data from the thresholding kernel's output port back to the global memory via the out GMIO port. Finally, the dimensions() API (**Lines 19, 21, 23, 24, 26**) explicitly configures the required buffer port sizes for all these connections in terms of total data samples (IP_IMG_H * IP_IMG_W) so the kernels know when a full block of data is ready to process.

The host code in the "**aie_top_all.cpp**"

```
1.      // File paths for input image, key image, and output image
2.      // Please set it to absolute path on your system
3.      const char *filename_ip = "<Path to file>/stego_image.png";
4.      const char *filename_key = "<Path to file>/key_image.png";
5.      const char *filename_out = "<Path to file>/decrypted_image_out_v.png";
```

Lines 1–5: Three file paths are defined – the stego (encrypted) input image, the key image, and the output (decrypted) image. Set the absolute paths before running the design.

```
6.      std::vector<unsigned char> image;    // GREY pixels (1 byte per pixel)
7.      std::vector<unsigned char> imageKey; // GREY pixels (1 byte per pixel)
```

Lines 6–7: Two std::vector<unsigned char> variables (image and imageKey) store the pixel data of the input and key images in grayscale format (1 byte per pixel).

```
8.      unsigned width = 0, height = 0;
9.      // Load input image and key image
10.     if (!load_decode_and_print_png(filename_ip, image, width, height)) {
11.         return -1;
12.     }
13.     if (!load_decode_and_print_png(filename_key, imageKey, width, height)) {
14.         return -1;
15.     }
```

Lines 8–15: **load_decode_and_print_png** reads **stego_image.png**, decodes the PNG into raw pixels, and fills image, width, and height. If loading fails, the program returns -1.

```
16.     // Set the size of input image and output image
17.     // Key image size = input image size
18.     int BLOCK_SIZE_in_BytesIP = INPUT_IMG_H * INPUT_IMG_W;
19.     int BLOCK_SIZE_in_BytesOUT = INPUT_IMG_H * INPUT_IMG_W;
```

Lines 16–19: BLOCK_SIZE_in_BytesIP and BLOCK_SIZE_in_BytesOUT are set to INPUT_IMG_H * INPUT_IMG_W, which is the total number of pixels in the image; since each pixel is 1 byte, this is also the buffer size in bytes. The image size we are dealing in this experiment in 256x256.

```
20.     uint8_t *dinArrayIP = (uint8_t *)GMIO::malloc(BLOCK_SIZE_in_BytesIP);
21.     for (int i = 0; i < BLOCK_SIZE_in_BytesIP; i++) {
22.         dinArrayIP[i] = image[i];
23.     }
24.
25.     uint8_t *dinArrayKey = (uint8_t *)GMIO::malloc(BLOCK_SIZE_in_BytesIP);
26.     for (int i = 0; i < BLOCK_SIZE_in_BytesIP; i++) {
27.         dinArrayKey[i] = imageKey[i];
28.     }
```

Line 20-28: GMIO::malloc allocates a buffer in global memory (dinArrayIP & dinArrayKey) for the input image and key data that will be sent to the AIE. A loop then copies each pixel from the image vector into dinArrayIP and dinArrayKey.

```
29.     uint8_t *doutArrayOUT = (uint8_t *)GMIO::malloc(BLOCK_SIZE_in_BytesOUT);
30.
31.     std::cout << "Graph initialized!!" << std::endl;
32.     accel.init();
```

Line 32: The accel.init() initializes the AIE graph (the hardware design that will run on the AIE tiles).

```
33.     std::cout << "Start data transfer to AIEs!!" << std::endl;
34.     accel.image.gm2aie_nb(dinArrayIP, BLOCK_SIZE_in_BytesIP);
35.     accel.key.gm2aie_nb(dinArrayKey, BLOCK_SIZE_in_BytesIP);
```

Lines 34-35: This starts a non-blocking transfers of the input and key data from global memory to the AIE kernels using **gm2aie_nb**.

```
36.     int iteration = (INPUT_IMG_H * INPUT_IMG_W) / (KERNEL_IP_IMG_H * KERNEL_IP_IMG_W);
37.     std::cout << "Graph execution started for " << iteration << " iterations!!" << std::endl;
38.     accel.run(iteration);
```

Line 36: iteration is computed as total image size divided by the size of one kernel block, so the graph runs for the required iterations to complete the computation.

Note: Here the **KERNEL_IP_IMG_H and KERNEL_IP_IMG_W** is set to 128, and the actual image size is set to 256x256. So, for your design to complete the decryption of the complete image. You will have to run the design 4 times (256*256/128*128 = 4).

```
39.     std::cout << "Start data transfer from AIEs!!" << std::endl;
40.     accel.out.aie2gm(doutArrayOUT, BLOCK_SIZE_in_BytesOUT);
41.
42.     std::cout << "Waiting for AIE graph to complete and transfer output data back!!" << std::endl;
43.     accel.out.wait();
44.
45.     savePNG(
46.         filename_out,
47.         std::vector<unsigned char>(doutArrayOUT, doutArrayOUT + BLOCK_SIZE_in_BytesOUT),
48.         width, height);
49.
50.     accel.end();
```

Line 40 initializes the output transaction. Line 43 waits until all the transfer is complete. At the end you use the savePNG() api to save the data to a PNG file.

6. Experiments

For the experiments mentioned in this section, you will be using a new input image. The image will be based on your group number. For example, if you are group 1, the filename you will use is key_group_01.png and stego_ group_01.png. The images for all groups are located here: /data/courses/class_cse494598cen571spring2026_aaror112/Lab4/Key_images

/data/courses/class_cse494598cen571spring2026_aaror112/Lab4/Stego

Your primary objective is to take a functional but unoptimized **scalar kernels** and transform it into a **high-performance vector application** distributed across multiple AI Engine tiles.

- **Optimize/Vectorize:** Use AIE vector instructions to reduce the overall kernel cycle counts. This implies modifying the AIE kernel files.
- **Parallelize:** Split the workload/computation of single kernel across multiple AIE cores to increase parallelism and reduce the overall compute time. This implies modifying the ADF graph file.
- **Functional Correctness:** Does the output image clearly reveal the hidden message?
- **Performance Gain:** What is the speedup ratio?
    - Speedup = (Baseline Scalar time (us)) / (Your Optimized Vector time (us))

1. Kernel Vectorization – 30 pts

The starter kernels are unoptimized **scalar** implementations. Your goal is to convert them to efficient **vector** kernels using the AIE APIs.

Your tasks:

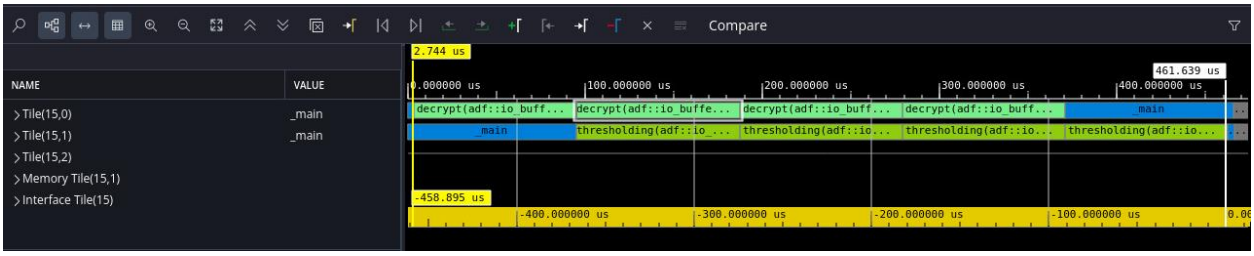
- Rewrite both kernels using AIE vector APIs (see the linked AMD vector-kernel example). Restructure your loops so the main work is done with vector operations (not scalar element-by-element code). Now, your steganography application has been vectorized.
- Run x86sim on the application to verify functional correctness.
- Run aiesim on the application to evaluate performance.

Note: The provided baseline design uses **scalar kernels**. As a result, **x86 simulation (x86sim)** will execute much faster and should be used first to verify functional correctness and visually confirm that the hidden message is correctly revealed. In contrast, **AI Engine simulation (aiesim)** executes significantly more slowly for the baseline design. A full baseline **aiesim** run at the original problem size (**256 × 256**) can take approximately **20 minutes**. You are encouraged to run this at least once to examine the trace and profile views and to understand the baseline performance characteristics. Since aiesim runtime may be long, especially if the assigned Sol machine is older, the kernel execution time and final graph execution time will be provided to you for the baseline design. You may use these values directly as the reference point for performance comparison. For faster iteration during development and debugging, you may temporarily reduce the kernel size to **64 × 64** elements. This allows quicker simulation turnaround while preserving the correctness of your optimization workflow. Be sure to restore the original problem size before reporting final performance results. Once you vectorize the kernel, your **aiesim** execution should be faster (~5min).

Here, are the results for the scalar run.

ID	FUNCTION NAME	NUMBER OF CALLS	TOTAL FUNCTION TIME (CYCLES)
1	decrypt	4	458804
2	main	1	123084
3	__cxa_finalize	1	43
4	_main_init	1	22
5	_fini	1	22

ID	FUNCTION NAME	NUMBER OF CALLS	TOTAL FUNCTION TIME (CYCLES)
1	thresholding	4	458804
2	main	1	123200
3	__cxa_finalize	1	43
4	_main_init	1	22
5	_fini	1	22



- Decrypt and Thresholding take **458804 cycles** for **4 iterations**
- Total execution from trace: **458.896 us**

In the lab report:

- Provide a snapshot from the Vitis window which includes green check mark next to Build in x86sim flow pane and output windows shows “Compilation complete” and “ERRORS:0”.
- Provide a snapshot from the Vitis window which includes green check mark next to Build in aiesim flow pane and output windows shows “Compilation complete” and “ERRORS:0”.
- Include snapshot of the ADF Graph (similar to Figure *20*)
- Include snapshot of AIE Array mapping (similar to Figure *21*)
- Include snapshots showing the cycles taken by both kernels (similar to Figure *24*, but for both kernels).
- Include snapshot of the Trace marking the start and end times (similar to Figure *25*)
- Include the screenshot of the output decrypted image
- Provide a table to compare the cycles consumed by the two kernels for the scalar (on AIE engines) and vectorized implementation (on AIE engines).
- Provide a table to compare the execution time of the complete application for the scalar (on AIE engines) and vectorized implementation (on AIE engines).
- Provide a speedup factor. Provide reasoning for the observed speed up.

2. Single buffer vs Double buffer – 20 pts

For this experiment, you will change the input and output ports to use **single buffering**. By default, if you do not specify anything, Vitis uses **double buffering**.

Note: You can force single buffering in the graph with the following call:

```
single_buffer(decrypt_k[0].in[0])
```

Your tasks:

- Update your code to use single buffering.
- Run x86sim first to verify functional correctness.
- Run aiseim to evaluate performance.

Note: Use your **vectorized kernels** from Experiment 1 for double buffer, since Vitis runs with double buffering as a default setting.

The ADF graph should show you if you have used single buffer or double buffer.

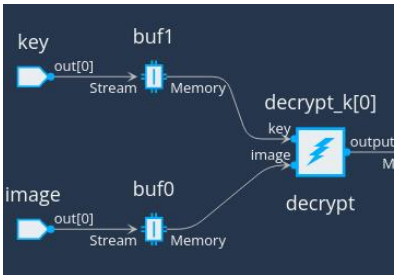


Figure 26: Single Buffer

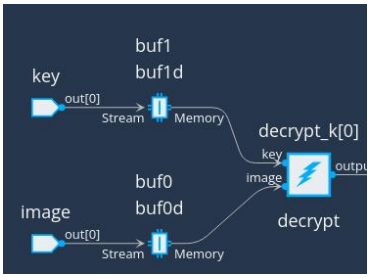


Figure 27: Double Buffer

In the lab report:

- Provide a snapshot from the Vitis window which includes green check mark next to Build in x86sim flow pane and output windows shows “Compilation complete” and “ERRORS:0”.
- Provide a snapshot from the Vitis window which includes green check mark next to Build in aiesim flow pane and output windows shows “Compilation complete” and “ERRORS:0”.
- Include snapshot of the ADF Graph (similar to Figure *20*)
- Include snapshot of AIE Array mapping (similar to Figure *21*)
- Include snapshots showing the cycles taken by both kernels (similar to Figure *24*, but for both kernels).
- Include snapshot of the Trace marking the start and end times (similar to Figure *25*)
- Include the screenshot of the output decrypted image
- Provide a table to compare the execution time of the complete application for the single-buffer and double-buffer implementations. Do you see any slowdown from single-buffering? If yes, where is the slowdown coming from?

3. Parallelizing your design – 50 pts

Once your kernels are fast, the next step is to make the design **wide** by spreading the work across multiple AIE tiles (more parallelism). In the previous experiences, the kernels were executed sequentially over four iterations. With **KERNEL_IP_IMG_H and KERNEL_IP_IMG_W** set to 128 and the image size at 256×256, the image is chunked into 4 chunks. As illustrated in Figure 28a, the vectorized implementation processes these four sections one after another, requiring four iterations to complete. However, the parallel implementation (Figure 28b) assigns each image section to a separate AIE tile. This allows all four sections to be processed simultaneously, enabling the kernels to run in parallel rather than sequentially.

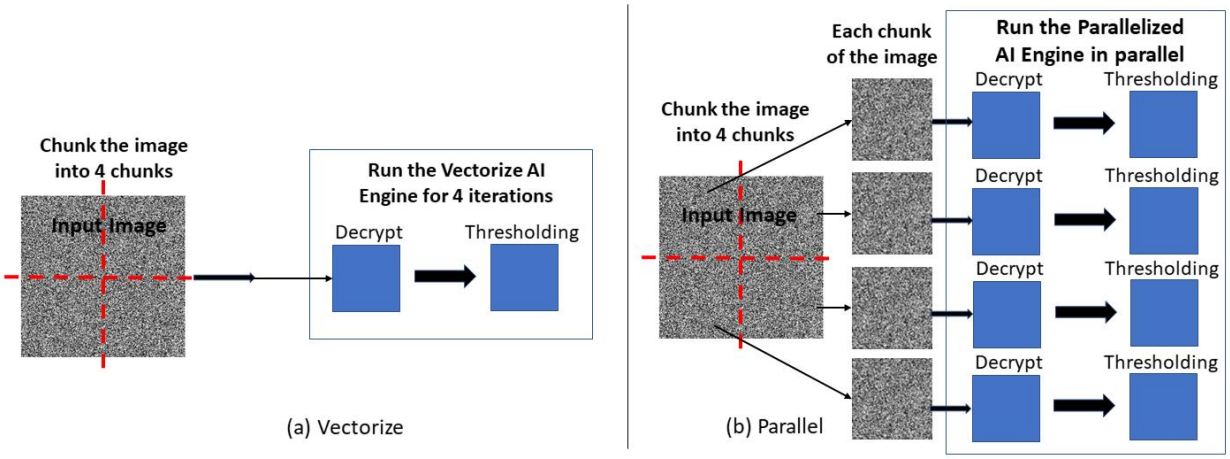


Figure 28: Parallelizing design

Your tasks:

- Partition the work of a single large kernel across multiple AIE tiles (for example, split the image into chunks).
- Update the ADF graph to map these kernels onto different physical AIE tiles (multi-tile mapping). **You are expected to replicate the two-AIE pipeline atleast 4 times (i.e. you should have a minimum of 8 AIEs being used in your solution).**

- Update the host code ("aie_top_all.cpp") accordingly. Split inputs into multiple buffers and collect outputs. Add more GMIO inputs/outputs. Change iteration count to 1.
- Run x86sim first to verify functional correctness.
- Run aiseim to evaluate performance.

In the lab report:

- Provide a snapshot from the Vitis window which includes green check mark next to Build in x86sim flow pane and output windows shows "Compilation complete" and "ERRORS:0".
- Provide a snapshot from the Vitis window which includes green check mark next to Build in aiesim flow pane and output windows shows "Compilation complete" and "ERRORS:0".
- Include snapshot of the ADF Graph (similar to Figure *20*)
- Include snapshot of AIE Array mapping (similar to Figure *21*)
- Include snapshots showing the cycles taken by both kernels (similar to Figure *24*, but for both kernels).
- Include snapshot of the Trace marking the start and end times (similar to Figure *25*)
- Include the screenshot of the output decrypted image
- Provide a table to compare the cycles consumed by the two kernels for the two-AIE vectorized implementation and multi-AIE parallel implementation.
- Provide a table to compare the execution time of the complete application for the two-AIE implementation and multi-AIE parallel implementation.
- Provide a speedup factor. Provide reasoning for the observed speed up.

4. Fine grained pipeline using Cascade <Extra Credit 10 pts>

AIE array hardware includes a special interface called **Cascade** that lets one AIE tile pass data directly to a neighboring AIE tile. For extra credit, use **Cascade** to stream partial results from the **decrypt** kernel to the **thresholding** kernel on each **inner-loop** iteration. This creates a fine-grained lockstep pipeline between the two kernels. To keep it simple, perform this experiment on the vectorized implementation using two AIEs only.

- Include the updated code in your submission.
- In your report, describe what you changed.
- Include snapshots showing the cycles taken by both kernels (similar to Figure *24*, but for both kernels).
- Include snapshot of the Trace marking the start and end times (similar to Figure *25*)
- Do you see any improvement in performance compared to Single/Double buffer vectorized implementation?

7. Deliverables

Please upload two files: 1. Zip file and 2. PDF file.

Create a zip file containing 3 folders (Experiment1, Experiment 2, Experiment 3). Name this file lab4_ <firstname1>_<lastname1>_<firstname2>_<lastname2>.zip. Upload this file to Canvas.

Also, create a report (PDF file. Name this file lab2_ <firstname1>_<lastname1>_<firstname2>_<lastname2>.pdf, and upload it to Canvas.

Code:

Provide all your code (source files (**kernels**, **connectivity graph**, and **host**), .cpp, output images, and etc..). Include any input/output or test files. Separate them into folders: Experiment 1, Experiment 2, Experiment 3. In each folder, provide a README file on how to compile/run/verify your code. Please copy and paste the log file for each Experiment separately as a text file in its directory. The TA should be able to reproduce the code.

Report:

Submit a **PDF report** with your experiment results. One file separated into sections. One section for each Experiment. Provide the information asked in each Experiment. When capturing screenshots of the graph and tile array for each experiment, make sure to zoom in beforehand so that the graph text and the placement of memory and cores within the tile array are clearly visible.

Explicitly in red color, mention anything you've done to get extra credit.

8. How will you be graded

- The correctness, completeness and performance of your solutions/code
- The correctness, completeness and clarity of your report
    - Please make sure you include the reasoning/commentary about your observations of the results. If you just put the results in the report without any discussion/commentary/conclusions, you will not get points.